Basic Vulnerability Identification Techniques

the process of examining a target environment to identify exploitable weaknesses. These weaknesses can occur in network services, operating systems, or applications. Before you can exploit anything, you need to know what is there and what is wrong with it.

Understanding the Attack Surface

The attack surface of a system is the total set of points where an attacker can attempt to interact with it. Every open port, running service, input field, and API endpoint is part of that surface.

What Makes Up an Attack Surface



Network - port accepting connections and every protocol in use

OS - user accounts, file permissions, scheduled tasks, and locally running services

Application - A web application receives input through URL parameters, form fields, HTTP headers, cookies, and uploaded files

A single target can present an attack surface at all three layers simultaneously. A thorough assessment accounts for each of them.

External vs. Internal

The external attack surface is everything reachable from outside the target's network. Publicly exposed web servers, VPN gateways, and mail servers all fall under this category.

The internal attack surface becomes relevant once an attacker has a foothold. File shares, management interfaces, database servers, and domain controllers that were invisible from the outside are now within reach.

Directing Your Efforts

Understanding the attack surface tells you where to focus. If a target exposes a web application with many dynamic pages and input fields alongside an SSH service running a current OpenSSH version with key-based authentication, the web application is the richer target.

Attack surface analysis also surfaces quick wins. A service on a non-standard port may be an old, forgotten deployment that has not been patched. An administrative interface exposed without authentication is an immediate finding. Mapping what is available before committing to detailed testing prevents you from wasting time on hardened components while weak ones go unexamined.

Service Enumeration and Banner Grabbing

Service enumeration is the process of discovering which services are listening on a target's open ports and gathering enough detail about each to support further testing.

Port Scanning with Nmap

knowing a port is open tells you very little on its own. Port 80 is conventionally used for HTTP, but nothing prevents an administrator from running SSH on it. Relying on port numbers alone leads to wrong assumptions.

The output shows the protocol, service name, and where Nmap can determine them, the software version for each open port. **This version information is the most valuable output of enumeration**. It is what lets you match a running service to known vulnerabilities.

Banner Grabbing

Banner grabbing is the process of connecting to a service and reading its initial response. Many services announce themselves on connection, providing a name and version string known as a banner.

Connecting to an SSH service with netcat:

nc 10.128.174.203 22

```
attacker@tryhackme:~$ nc 10.128.174.203 22
SSH-2.0-OpenSSH_7.6p1 Ubuntu-4ubuntu0.3
```

That single string reveals the OpenSSH version and the underlying operating system. It is enough to start searching for known vulnerabilities against that version.

Connecting to an SMTP service might return:

nc 10.128.174.203 25

```
attacker@tryhackme:~$ nc 10.128.174.203 25
220 mail.target.local ESMTP Postfix (Ubuntu)
```

You can perform banner grabbing manually with netcat ortelnet, or automate it through Nmap's version detection probes.

When Banners Are Absent or Misleading

Not every service gives a useful banner. Some suppress version information entirely, returning only a generic greeting.

Banners can also be deliberately changed. An administrator aware that attackers use banner information might alter the version string. However, this is uncommon outside honeypot environments. Other indicators, such as response behaviour and supported protocol features, often reveal the real identity of the service regardless.

Building a Service Inventory

As you enumerate each port, build an inventory. For each service, record the port number, protocol, software name and version, and any extra details revealed by scripts or manual inspection.

Port	Service	Version	Notes
22	SSH	OpenSSH 7.6p1	Ubuntu banner
80	HTTP	Apache 2.4.29	Default page present
445	SMB	Samba 4.7.6	Signing not required
3306	MySQL	5.7.33	Remote connections enabled

Matching Services to Known Exploits

With a service inventory in hand, the next step is to determine whether any of the discovered services contain known weaknesses. This means cross-referencing version information against public vulnerability databases and working out whether the results actually apply to the target.

Public Vulnerability Databases

can use https://www.cve.org/ or https://nvd.nist.gov/

You can search Exploit-DB from the command line using searchsploit, which queries a local copy of the database:

searchsploit apache 2.4.29

```
root@ip-10-130-127-34:~# searchsploit apache 2.4.29
---------------------------------------------- ---------------------------------
 Exploit Title                                |  Path
---------------------------------------------- ---------------------------------
Apache + PHP < 5.3.12 / < 5.4.2 - cgi-bin Rem | php/remote/29290.c
Apache + PHP < 5.3.12 / < 5.4.2 - Remote Code | php/remote/29316.py
Apache 2.4.17 < 2.4.38 - 'apache2ctl graceful | linux/local/46676.php
Apache CXF < 2.5.10/2.6.7/2.7.4 - Denial of S | multiple/dos/26710.txt
Apache mod_ssl < 2.8.7 OpenSSL - 'OpenFuck.c' | unix/remote/21671.c
Apache mod_ssl < 2.8.7 OpenSSL - 'OpenFuckV2. | unix/remote/47080.c
Apache mod_ssl < 2.8.7 OpenSSL - 'OpenFuckV2. | unix/remote/764.c
Apache OpenMeetings 1.9.x < 3.1.0 - '.ZIP' Fi | linux/webapps/39642.txt
Apache Tomcat < 5.5.17 - Remote Directory Lis | multiple/remote/2061.txt
Apache Tomcat < 6.0.18 - 'utf8' Directory Tra | multiple/remote/6229.txt
Apache Tomcat < 6.0.18 - 'utf8' Directory Tra | unix/remote/14489.c
Apache Tomcat < 9.0.1 (Beta) / < 8.5.23 / < 8 | jsp/webapps/42966.py
Apache Tomcat < 9.0.1 (Beta) / < 8.5.23 / < 8 | windows/webapps/42953.txt
Apache Xerces-C XML Parser < 3.1.2 - Denial o | linux/dos/36906.txt
Webfroot Shoutbox < 2.32 (Apache) - Local Fil | linux/remote/34.pl
---------------------------------------------- ---------------------------------
Shellcodes: No Results
```

This returns matching entries along with the file path to the exploit code on your system.

GitHub is another valuable source, and often the fastest one. When a high-profile CVE is disclosed, proof-of-concept exploit code frequently appears on GitHub within hours or days. Security researchers and offensive tooling developers publish working PoCs, sometimes before the vulnerability even makes it into Exploit-DB

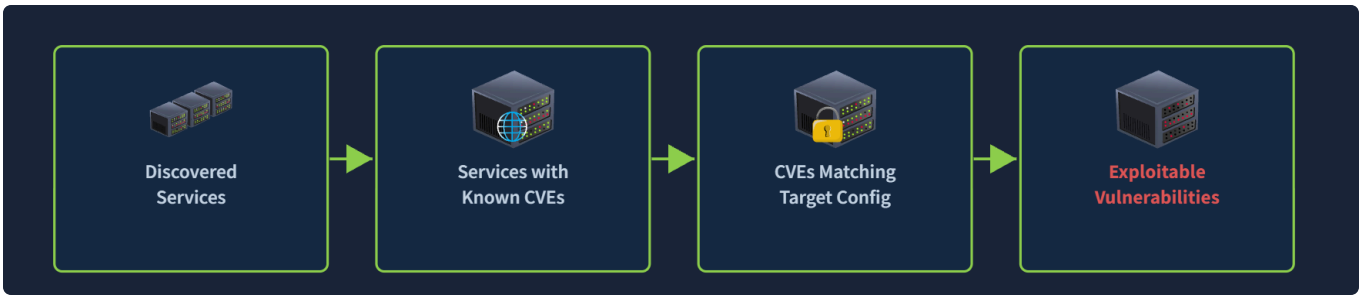
A typical workflow after enumeration:

1. Take the first service and version from your inventory.
2. Search the NVD or Exploit-DB for that software name and version.
3. Check GitHub for the CVE identifier to find any proof-of-concept code or tooling.
4. Review the results, noting any CVEs affecting the exact version or a range that includes it.
5. Assess each result for relevance and exploitability.
6. Repeat for every service in the inventory.

Severity Ratings

Most CVE entries carry a CVSS score, rating severity from 0.0 to 10.0. The score accounts for the attack vector (network vs local), whether authentication is required, and the impact on confidentiality, integrity, and availability.

The Gap Between "Vulnerable" and "Exploitable"



A CVE might describe a buffer overflow in a particular version of a web server. However, if the target has applied a patch without updating the version string, the vulnerability is no longer there. A flaw that requires a specific module to be enabled is irrelevant if the target's configuration does not include that module.

Network-level controls might block the traffic needed to trigger a vulnerability. An exploit that requires valid credentials is useless if you have not obtained any. A vulnerability that looks promising in a database can turn out to be a dead end once you account for what is actually running on the target.

Automated Vulnerability Scanners

Tools such as Nessus(opens in new tab), OpenVAS(opens in new tab), and Nikto(opens in new tab) automate the process of matching services against vulnerability databases. They run their own enumeration and compare results against known issues, producing a report of potential vulnerabilities.

Identifying Web Application Vulnerabilities

Web applications accept user input through many channels and interact with backend databases and filesystems. Identifying vulnerabilities in them requires a different approach from scanning network services. Rather than matching version numbers to CVE entries, you are probing how the application behaves and looking for patterns that point to exploitable flaws.

Establishing a Baseline

Before testing, spend time using the application as a normal user would. Browse every page, submit forms with legitimate data, and observe how the application responds. This gives you a map of its functionality and a baseline of normal behaviour.

You can also use Burp Suite's proxy running while you browse so that every request and response gets captured in the site map. Pay attention to URL structures, query parameters, form fields, cookies, and hidden fields in the HTML source.

Recognising Injection Points

Injection vulnerabilities occur when user-supplied input ends up in a command, query, or expression that the application processes without proper sanitisation

Imagine a web application that displays user profiles based on a numeric ID in the URL:

```
https://target.local/profile?id=15
```

A legitimate request returns the expected profile page. The backend is constructing something like:

```
SELECT * FROM users WHERE id = '15'
```

Now submit a value that breaks the query:

```
https://target.local/profile?id=15'
```

The query becomes:

```
SELECT * FROM users WHERE id = '15''
```

Spotting Access Control Weaknesses

Access control flaws arise when an application fails to enforce restrictions on what authenticated users can do. An attacker can often escalate privileges or access other users' data with nothing more than a modified request.

Imagine your account profile is at:

```
https://target.local/api/user/1042
```

Change the ID:

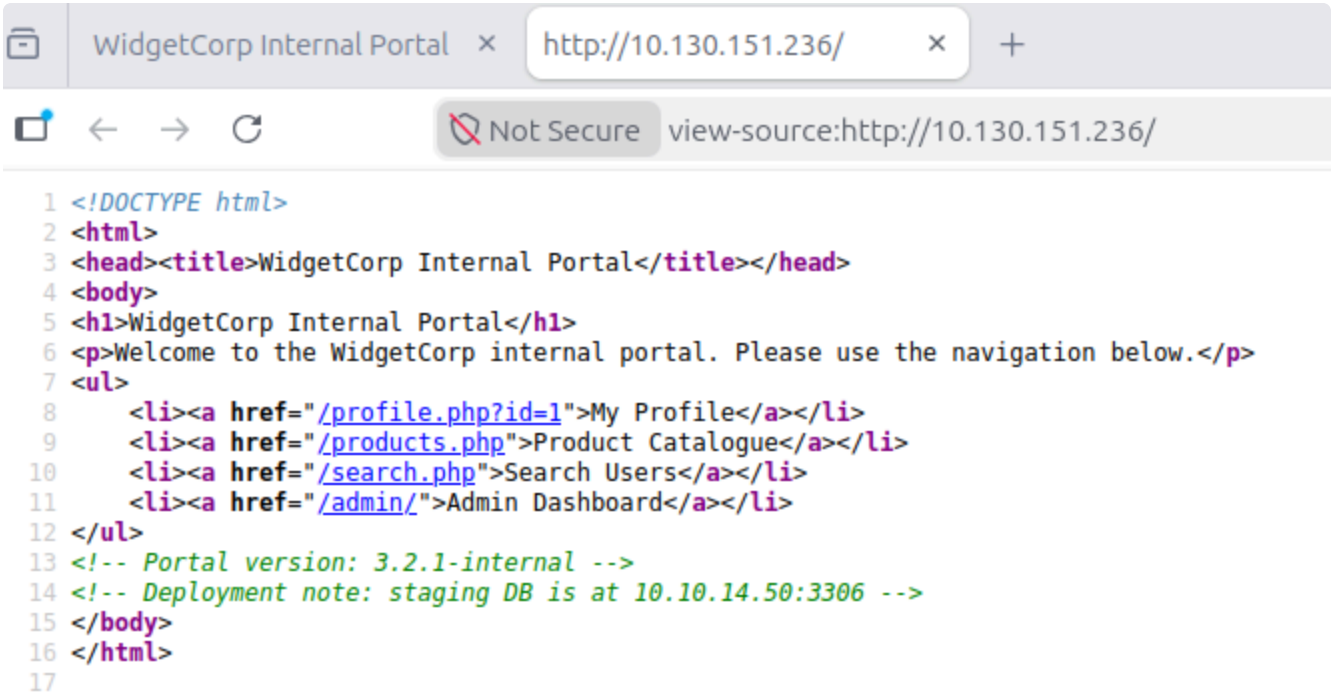
```
https://target.local/api/user/1043
```

Information Disclosure

Information disclosure is any situation where the application reveals data that helps an attacker advance. This can include technical details about infrastructure, internal paths, software versions, or in more serious cases, credentials and API keys.

A Server: Apache/2.4.29 (Ubuntu) header tells an attacker exactly what to search for in a vulnerability database. A stack trace that includes a database connection string is far more severe.

Make a habit of inspecting response headers and page source for every page you visit. Submit invalid input and observe whether error messages are generic or detailed. Request common paths like /robots.txt, /.git/, and /backup/ to check for exposed files. These checks take very little time and frequently produce findings that shape the rest of the engagement.



Page source revealed a comment with an IP of the staging DB

Identifying System and Network Vulnerabilities

Not all vulnerabilities live in web applications. Network services and operating system configurations have their own weaknesses, many of which are straightforward to find.

Default and Weak Credentials

Many services ship with well-known default usernames and passwords. Administrators do not always change them after deployment. Databases, network appliances, management consoles, and IoT devices are particularly prone to this.

When you find a service that requires authentication, check whether the default credentials work. The DefaultCreds-Cheat-Sheet https://github.com/ihebski/DefaultCreds-cheat-sheet repository and vendor documentation list factory-set credentials for thousands of products.

If the defaults have been changed but the service allows remote login attempts, you can test a short list of common weak passwords. For SSH:

hydra -l admin -P /usr/share/wordlists/rockyou.txt ssh://10.130.151.236

SMB Misconfigurations

The Server Message Block (SMB) protocol is used widely in Windows environments for file sharing and inter-process communication. Misconfigurations are common.

The first check is whether null session access is allowed. A null session lets an unauthenticated user connect and enumerate shares, user accounts, and group memberships:

smbclient -L //10.130.151.236 -N

```
root@ip-10-130-87-94:~# smbclient -L //10.130.151.236 -N

        Sharename       Type      Comment
        ---------       ----      -------
        shared          Disk      Internal shared documents
        IPC$            IPC       IPC Service (WidgetCorp File Server)
SMB1 disabled -- no workgroup available
```

The -N flag suppresses the password prompt. If the target returns a share list, you have unauthenticated access to at least the listing. From there, try connecting to each share and check whether any allow read or write access without credentials.

Another common weakness is SMB signing not being required. When signing is disabled or optional, an attacker on the same network segment can perform relay attacks, intercepting authentication requests and forwarding them to other services:

nmap --script smb2-security-mode -p 445 10.130.151.236

```
root@ip-10-130-87-94:~# nmap --script smb2-security-mode -p 445 10.130.151.236
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-08-20 11:11 UTC
Nmap scan report for ip-10-130-151-236.eu-west-3.compute.internal (10.130.151.23
6)
Host is up (0.00038s latency).

PORT    STATE SERVICE
445/tcp open  microsoft-ds

Host script results:
| smb2-security-mode:
|   3:1:1:
|_    Message signing enabled but not required

Nmap done: 1 IP address (1 host up) scanned in 1.48 seconds
```

If the output shows that message signing is enabled but not required, the service is vulnerable to relay.

FTP Misconfigurations

FTP is an older file transfer protocol still found in many environments, especially on legacy systems. The most common misconfiguration is anonymous access:

```
ftp 10.130.151.236
Name: anonymous
Password: anything@here.com
```

```
root@ip-10-130-87-94:~# ftp 10.130.151.236
Connected to 10.130.151.236.
220 (vsFTPd 3.0.5)
Name (10.130.151.236:root): anonymous
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
ftp>
```

If the server accepts the connection, look through the directory listing for sensitive files. FTP servers with anonymous access sometimes expose configuration files, backups, or credentials. Even if the files are not directly exploitable, the presence of an anonymous FTP server is worth noting.

Beyond anonymous access, check whether the service transmits credentials in plaintext. If it does, an attacker with network access can capture them through packet sniffing.

```
dr-xr-xr-x    2 ftp      ftp          4096 Mar 17 08:03 pub
226 Directory send OK.
ftp> cd /pub
250 Directory successfully changed.
ftp> ls
229 Entering Extended Passive Mode (|||40069|)
150 Here comes the directory listing.
-rw-r--r--    1 ftp      ftp           154 Mar 17 08:03 README.txt
-rw-r--r--    1 ftp      ftp           246 Mar 17 08:03 maintenance-schedule.txt
-rw-r--r--    1 ftp      ftp           387 Mar 17 08:03 network-map.txt
226 Directory send OK.
ftp> ls -la
229 Entering Extended Passive Mode (|||40020|)
150 Here comes the directory listing.
dr-xr-xr-x    2 ftp      ftp          4096 Mar 17 08:03 .
dr-xr-xr-x    3 ftp      ftp          4096 Mar 17 08:03 ..
-rw-r--r--    1 ftp      ftp           154 Mar 17 08:03 README.txt
-rw-r--r--    1 ftp      ftp           246 Mar 17 08:03 maintenance-schedule.txt
-rw-r--r--    1 ftp      ftp           387 Mar 17 08:03 network-map.txt
226 Directory send OK.
```

```
ftp> get network-map.txt
local: network-map.txt remote: network-map.txt
229 Entering Extended Passive Mode (|||40087|)
150 Opening BINARY mode data connection for network-map.txt (387 bytes).
100% |***********************************|   387        1.54 MiB/s    00:00 ETA
226 Transfer complete.
387 bytes received in 00:00 (529.31 KiB/s)
ftp> cat network-map.txt
?Invalid command.
ftp> !cat network-map.txt
WidgetCorp Internal Network Map (Q3 2024)
==========================================
Web Server:       10.10.14.10 (this machine)
Database Server:  10.10.14.50 (MariaDB, PostgreSQL)
Domain Controller: 10.10.14.1 (dc01.widgetcorp.local)
Backup Server:    10.10.14.60 (rsync, SSH)
Dev Environment:  10.10.14.100 (Docker host)

Default credentials for dev environment: devops / D3vOps2024!
ftp>
```

Triaging and Documenting Findings

By this point in an engagement, you will have collected a range of potential vulnerabilities across different services and applications. Not all of them carry the same weight. Triage is the process of evaluating each finding, assessing what it actually enables, and deciding which ones to pursue first.

Assessing Impact

The impact of a vulnerability depends on what an attacker gains by exploiting it. A remote code execution flaw on an externally facing web server is a fundamentally different finding from an information disclosure issue on an internal monitoring dashboard.

The most impactful vulnerabilities grant direct access to a system, enable privilege escalation, or expose sensitive data such as credentials. A finding that reveals database credentials might not be exploitable on its own, but it could unlock access to a database server that was otherwise unreachable.

Where the affected system sits in the environment matters too. A vulnerability on a domain controller has far greater consequences than the same vulnerability on a standalone workstation, because compromising the domain controller can hand an attacker control over the entire Active Directory environment.

Prioritising

1 Tier 1

Immediate investigation

- Unauthenticated remote code execution
- Default credentials on critical services
- Administrative functions exposed without auth
- Unpatched services with known exploits

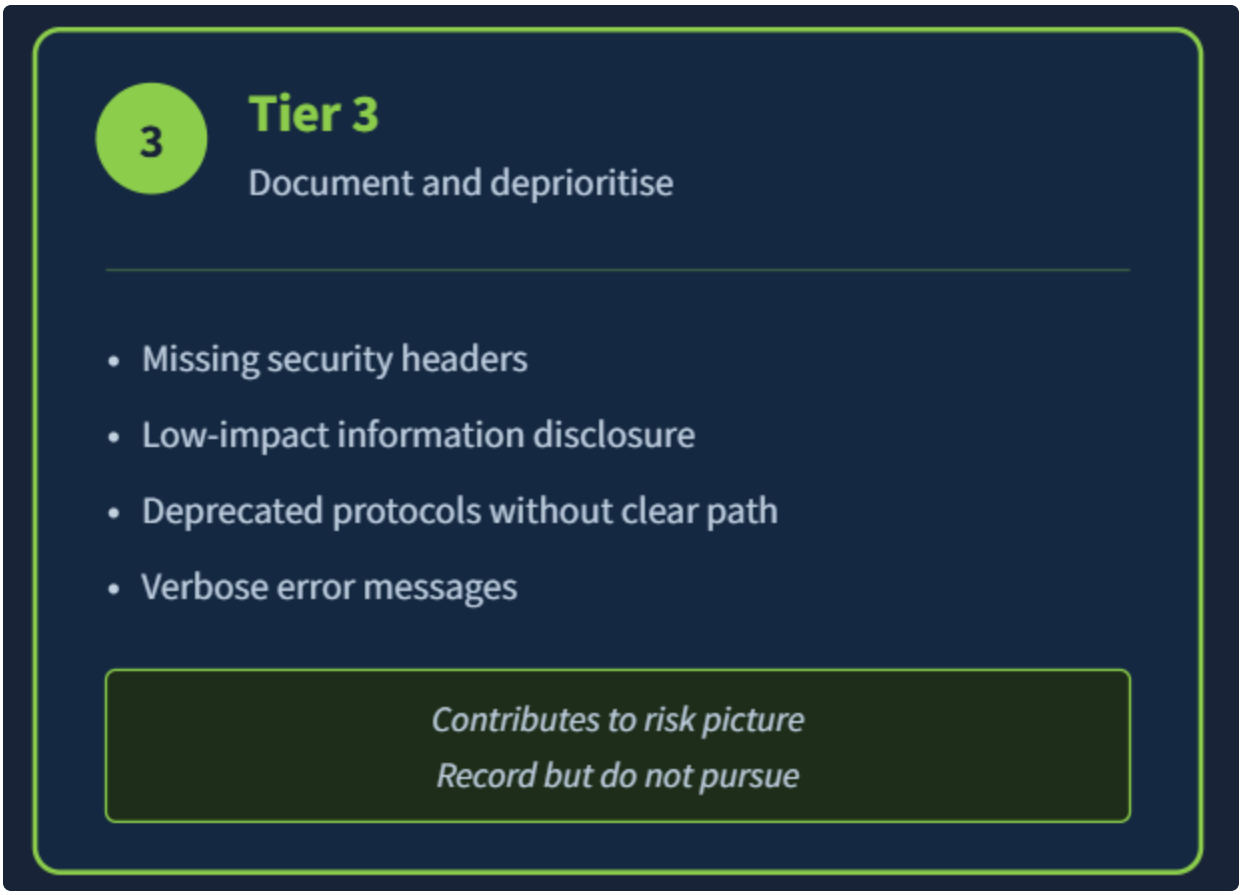
Highest return on investigation time
Investigate and document first

2 Tier 2

Requires additional work

- Blind SQL injection requiring extraction
- CVE matches but patch status unclear
- IDOR needing a second user session
- Chained weaknesses across endpoints

Significant payoff but more time required
Investigate once Tier 1 is resolved



Documentation

Every finding should be recorded as you go, not reconstructed from memory afterwards. For each one, note the affected host or component, the type of vulnerability, the evidence (the request and response, for instance), the steps to reproduce it, and an assessment of the impact.

Avoid vague descriptions. **"The web application may be vulnerable to injection"** is not useful. **"The `q` parameter on `GET /search.php` returns a MySQL syntax error when a single quote is submitted"** gives anyone reading the note enough to understand and reproduce the issue.

Screenshots and request/response pairs from Burp Suite make particularly good evidence. They provide an unambiguous record of what you observed.

Practical

in FTP can use anonymous to log in if its not secure it will log you in here you can read directories using ls or ls -la

this can give access to files. with:
get [filename]

then !cat [filename]

If an SMB share is available you can do
!smbclient //[ip]/[sharename] -N

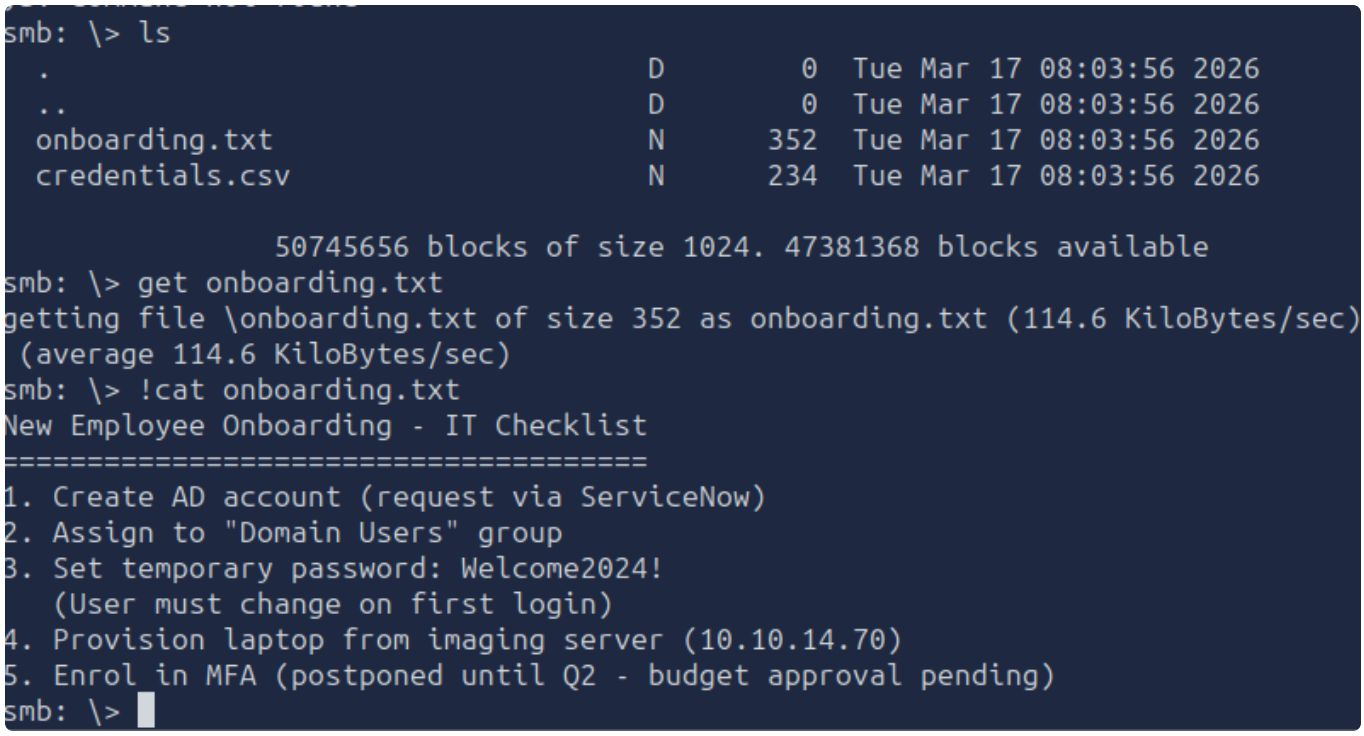
```
ftp> !smbclient //10.130.151.236/shared -N
Try "help" to get a list of possible commands.
smb: \>
```

then can do the same in the share

his can give access to files. with:
get [filename]

then !cat [filename]

```
root@ip-10-128-100-23:~# ftp 10.128.167.247
Connected to 10.128.167.247.
220 (vsFTPd 3.0.5)
Name (10.128.167.247:root): anonymous
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
ftp> smbclient //10.128.167.247/share -N
?Invalid command.
ftp> !smbclient //10.128.167.247/share -N
tree connect failed: NT_STATUS_BAD_NETWORK_NAME
ftp> !smbclient //10.128.167.247/shared -N
Try "help" to get a list of possible commands.
smb: \> ls -la
NT_STATUS_NO_SUCH_FILE listing \-la
smb: \> ;s
;s: command not found
smb: \> ls
```



Further Resources

OWASP Testing Guide https://owasp.org/www-project-web-security-testing-guide/ for a comprehensive web application vulnerability identification methodology

NIST National Vulnerability Database (NVD) https://nvd.nist.gov/ for searching CVE entries and understanding CVSS scoring

Exploit-DB https://www.exploit-db.com/ for publicly available exploit code associated with known vulnerabilities